

DOCUMENTO DE ANALISIS Y RESULTADOS

Data Wrangling: Limpieza, Preparacion y Segmentacion de Datos
Actividad de Aprendizaje No. 5 - Modulo: Ciencia de Datos con Python

Programa: Analisis y Desarrollo de Software (ADSO)

Empresa del caso: DataAnalytics Colombia S.A.S.

Instructor: Efren Moreno Valoyes

1. Introduccion

El Data Wrangling es el proceso mediante el cual un conjunto de datos crudo, que normalmente contiene errores, inconsistencias, valores faltantes y formatos distintos, se transforma en un conjunto de datos limpio, confiable y listo para ser utilizado en analisis, indicadores o toma de decisiones. Este proceso incluye diagnosticar la calidad de los datos, identificar los problemas existentes, limpiarlos y prepararlos, y finalmente segmentarlos de acuerdo con criterios de negocio.

La empresa DataAnalytics Colombia S.A.S. recibio una base de datos de clientes proveniente de diferentes sistemas. Dicha base presenta problemas de calidad tipicos de un entorno real: registros duplicados, datos faltantes, nombres de ciudades escritos de formas distintas, categorias inconsistentes, edades invalidas y valores negativos. El presente documento describe el proceso de Data Wrangling desarrollado en Python con la libreria Pandas para diagnosticar, limpiar, preparar y segmentar dicha informacion, dejandola lista para su uso en analisis, indicadores y campanas comerciales.

2. Diagnostico Inicial de los Datos

El archivo original clientes_originales_data_wrangling.csv fue cargado mediante Pandas sin ser modificado en ningun momento. Sobre esta copia en memoria se realizo un diagnostico inicial, cuyos resultados mas relevantes se resumen a continuacion.

Aspecto evaluado	Resultado
Cantidad total de registros	123
Cantidad de columnas	9
Nombres de columnas	ID, Nombre, Edad, Ciudad, Genero, Categoria, Compras, ValorCompra, Estado
Valores nulos totales	6 (uno en cada una de las columnas Edad, Ciudad, Genero, Categoria, ValorCompra y Estado)
Registros duplicados	2 registros totalmente duplicados
Edad minima / maxima	-3 / 104 (valores fuera de rango logico)
ValorCompra minimo / maximo	-1.500.000 / 7.200.000 (valores negativos invalidos)

Se revisaron además los valores únicos de las columnas categóricas (Ciudad, Género, Categoría y Estado), lo que permitió detectar múltiples formas de escritura para un mismo dato, como se detalla en la siguiente sección.

3. Problemas de Calidad Encontrados

Problema	Detalle encontrado en los datos
Valores nulos	1 valor nulo en cada una de las columnas Edad, Ciudad, Género, Categoría, ValorCompra y Estado.
Registros duplicados	2 registros idénticos en todas sus columnas (ID 13 y 58 aparecen dos veces).
Espacios innecesarios	Valores como " Cartagena ", " activo " y " Estandar " con espacios al inicio o al final.
Ciudades inconsistentes	La misma ciudad escrita de varias formas, por ejemplo: Medellín, medellin, MEDELLIN, Medellín (con y sin tilde).
Género inconsistente	Valores como Masculino, masculino, M, Hombre; y Femenino, femenino, F, Mujer, FEMENINO.
Categoría inconsistente	Valores como Premium, premium, PREMIUM; Estandar, estandar, Estandar; Basica, BASICA, basica; Corporativo, CORPORATIVO.
Estado inconsistente	Valores como Activo, activo, ACTIVO; Inactivo, inactivo, INACTIVO, algunos con espacios adicionales.
Edades inválidas	3 registros con edad fuera de rango lógico: 15 años (menor de edad), 104 años y -3 años (valor imposible).
Compras en cero	3 registros con Compras = 0. Se documenta como hallazgo, pero no se considera un error: un cliente registrado sin compras aun es un dato válido.
Valores de compra negativos	3 registros con ValorCompra negativo, por ejemplo -250.000, lo que indica un probable error de digitación en el signo.

4. Proceso de Limpieza y Preparación

Para cada problema identificado se aplicó una estrategia de limpieza específica, siempre trabajando sobre una copia del DataFrame original (el archivo CSV original nunca fue modificado ni sobrescrito). A continuación se explica cada estrategia y su justificación.

4.1 Registros duplicados

Se eliminaron con el metodo `drop_duplicates()`, que conserva unicamente la primera aparicion de cada registro repetido. Esto evita contar dos veces al mismo cliente en analisis posteriores.

4.2 Espacios innecesarios

Se eliminaron los espacios en blanco al inicio y al final de todas las columnas de texto mediante `str.strip()`, de forma que valores como " Cartagena " y "Cartagena" se reconozcan como el mismo dato.

4.3 Normalizacion de texto (Ciudad, Genero, Categoria, Estado)

Cada columna categorica se convirtio primero a mayusculas con `str.upper()`, y luego se aplico el metodo `replace()` con un diccionario que asigna cada variante encontrada a su forma oficial (por ejemplo, MEDELLIN, medellin y Medellin se convierten todas en Medellin, conservando la tilde correcta en el resultado final).

4.4 Tratamiento de valores nulos

Para las columnas numericas Edad y ValorCompra, los valores nulos se rellenaron con la mediana de cada columna mediante `fillna()`, ya que la mediana es una medida robusta que no se distorsiona por valores extremos. Para las columnas categoricas Ciudad, Genero, Categoria y Estado, los nulos se rellenaron con la etiqueta "Desconocido", de forma que el registro no se pierda pero tampoco se invente un dato que no se conoce con certeza.

4.5 Edades invalidas

Las edades menores a 18 anios, mayores a 100 anios o negativas se identificaron con una condicion booleana y se reemplazaron por la mediana de las edades validas, utilizando el metodo `loc[]` para modificar unicamente las filas afectadas sin alterar el resto del DataFrame.

4.6 Valores de compra invalidos

Los valores negativos de ValorCompra se corrigieron tomando su valor absoluto, bajo el supuesto de que corresponden a un error de digitacion en el signo y no a una compra negativa real, evitando asi descartar registros completos por un solo campo erroneo.

4.7 Verificacion posterior a la limpieza

Una vez aplicadas todas las correcciones, se volvio a ejecutar el mismo diagnostico inicial sobre el DataFrame limpio. El resultado confirmo que no quedaron valores nulos, no quedaron registros duplicados, y las columnas categoricas quedaron con un unico valor estandar por categoria (por ejemplo, Ciudad quedo unicamente con Barranquilla, Bogota, Cali, Cartagena, Medellin, Quibdo y Desconocido).

5. Filtros Condicionales Aplicados

Sobre el conjunto de datos ya limpio se aplicaron los siguientes seis filtros, combinando condiciones simples y multiples mediante los operadores logicos de Pandas & (AND), | (OR) y ~ (NOT).

Filtro	Condicion aplicada
1. Clientes mayores de edad	<code>df[df["Edad"] >= 18]</code>
2. Clientes de Medellin	<code>df[df["Ciudad"] == "Medellin"]</code>
3. Clientes de alto valor	<code>df[df["ValorCompra"] > 3000000]</code>
4. Medellin y mayores de 25 anios	<code>df[(df["Edad"] > 25) & (df["Ciudad"] == "Medellin")]</code>
5. Medellin o Cali	<code>df[(df["Ciudad"] == "Medellin") (df["Ciudad"] == "Cali")]</code>
6. Clientes que no son de Bogota	<code>df[~(df["Ciudad"] == "Bogota")]</code>

6. Segmento con el Metodo isin()

Como lo exige la actividad, se construyo un segmento adicional utilizando obligatoriamente el metodo `isin()`, el cual permite verificar si el valor de una columna pertenece a una lista de valores permitidos, evitando encadenar varias condiciones con el operador OR:

```
df[df["Ciudad"].isin(["Medellin", "Cali", "Bogota", "Quibdo"])]
```

Este filtro selecciona todos los clientes que residen en alguna de las cuatro ciudades indicadas.

7. Segmentacion Comercial

Se construyeron cinco segmentos comerciales sobre el conjunto de datos limpio, cada uno con una regla de negocio distinta:

Segmento	Regla de negocio
A. Clientes premium	ValorCompra mayor a 5.000.000
B. Clientes jovenes	Edad entre 18 y 25 años (metodo between)
C. Ciudades principales	Ciudad en Medellin, Cali o Bogota (metodo isin)
D. Clientes activos	Estado distinto de Inactivo (operador ~)
E. Clientes de alto potencial	Edad entre 25 y 50, mas de 5 compras, ValorCompra mayor a 2.000.000 y Estado Activo (metodo query)

Cada segmento se exporto como una hoja independiente dentro del archivo reporte_segmentacion.xlsx, junto con una hoja Resumen que indica la cantidad de clientes que cumple cada regla.

8. Investigacion de Metodos Adicionales de Pandas

Como parte de la investigacion obligatoria, se documentan a continuacion diez metodos de Pandas utilizados en el proyecto, indicando que hacen, su sintaxis, el problema que resuelven y como fueron aplicados en el codigo.

Metodo	Sintaxis	Que hace / problema que resuelve	Uso en el proyecto
dropna()	df.dropna()	Elimina las filas que contengan al menos un valor nulo. Evita que datos incompletos afecten calculos posteriores.	Se uso como verificacion final de seguridad, luego de tratar todos los nulos conocidos con fillna().
fillna()	df["col"].fillna(valor)	Reemplaza los valores nulos por un valor especifico. Evita perder registros completos por un solo dato faltante.	Se uso para rellenar Edad y ValorCompra con la mediana, y las columnas de texto con "Desconocido".
drop_duplicates()	df.drop_duplicates()	Elimina filas que son exactamente iguales a otra fila anterior. Evita contar dos veces al mismo cliente.	Se aplico al inicio de la limpieza para eliminar los 2 registros duplicados detectados.
replace()	serie.replace({"A": "B"})	Reemplaza los valores que coinciden con las llaves de un	Se uso para normalizar Ciudad, Genero, Categoria

Metodo	Sintaxis	Que hace / problema que resuelve	Uso en el proyecto
		diccionario. Unifica distintas formas de escribir el mismo dato.	y Estado a su forma oficial.
astype()	df["col"].astype(int)	Convierte una columna a un tipo de dato especifico. Corrige el tipo de dato luego de la limpieza.	Se aplico a la columna Edad para dejarla como numero entero tras rellenar los nulos.
str.strip()	df["col"].str.strip()	Elimina espacios en blanco al inicio y al final de un texto. Evita que Pandas trate el mismo dato como distinto por espacios extra.	Se aplico a todas las columnas de texto antes de normalizarlas.
str.upper()	df["col"].str.upper()	Convierte el texto de una columna a mayusculas. Permite comparar texto sin importar mayusculas o minusculas.	Se uso como paso previo a replace() para normalizar las columnas categoricas.
between()	df["col"].between(a, b)	Verifica si un valor numerico esta dentro de un rango, incluyendo los limites. Es mas legible que combinar los operadores >= y <= con &.	Se uso para construir el Segmento_Joven (edad entre 18 y 25 años).
query()	df.query("col > 5")	Filtra un DataFrame escribiendo la condicion como una cadena de texto. Facilita leer filtros con varias condiciones combinadas.	Se uso para construir el segmento Alto_Potencial, que combina cuatro condiciones distintas.
loc[]	df.loc[condicion, "col"] = valor	Selecciona y permite modificar filas y columnas segun una condicion booleana. Permite corregir unicamente las filas invalidas sin alterar las demas.	Se uso para reemplazar las edades y los valores de compra invalidos.

9. Conclusiones

- El conjunto de datos original presentaba multiples problemas tipicos de fuentes de datos reales: duplicados, nulos, inconsistencias de formato y valores fuera de rango logico.

- Aplicar un proceso reproducible con funciones separadas por fase (diagnostico, limpieza, filtros, segmentacion y exportacion) permite que el proceso se pueda auditar, repetir y modificar facilmente.
- El uso de logica booleana (&, | y ~) junto con el metodo isin() permitio construir filtros y segmentos comerciales claros y precisos.
- Los archivos clientes_limpios.csv y reporte_segmentacion.xlsx quedan listos para ser utilizados por el equipo de analisis e indicadores de la empresa DataAnalytics Colombia S.A.S., sin haber alterado en ningun momento el archivo original.